ЧАСТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ

«СУРГУТСКИЙ КОЛЛЕДЖ ПРЕДПРИНИМАТЕЛЬСТВА» (ЧПОУ «СКП»)

КУРСОВОЙ ПРОЕКТ

Тема: «Проектирование и разработка модуля регистрации и обработки программных ошибок»

ПО ПМ.01 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ДЛЯ КОМПЬЮТЕРНЫХ СИСТЕМ

МДК.01.02 ПРИКЛАДНОЕ ПРОГРАММИРОВАНИЕ

Специальность 09.02.07 «Информационные системы и программирование»

Выполнил:
студент группы ИСиП/23-1
Цоколов Даниил Дмитриевич

Руководитель:
Крючков Виталий Владимирович

Сургут, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РЕГИСТРАЦИИ И ОБРАБОТКИ ПРОГРАММНЫХ ОШИБОК В АВТОМАТИЗИРОВАННЫХ СИСТЕМАХ 5

1.1 Понятие и классификация программных ошибок.....5

1.2 Обработка ошибок и журналирование как элементы защиты информации 7

1.3 Механизмы обработки ошибок и журналирования на платформе C#/.NET 9

2 АНАЛИЗ СЕРВЕРНОЙ ЧАСТИ МЕССЕНДЖЕРА AIRGRAP И СОСТОЯНИЯ МОДУЛЯ ОБРАБОТКИ ОШИБОК.....11

2.1 Характеристика автоматизированной системы и применённых средств разработки.....11

2.2 Анализ процессов регистрации и аутентификации пользователей. .13

2.3 Анализ существующего состояния модуля регистрации и обработки ошибок..... 15

3 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА МОДУЛЯ РЕГИСТРАЦИИ И ОБРАБОТКИ ПРОГРАММНЫХ ОШИБОК.....17

3.1 Требования к модулю.....17

3.2 Проектирование архитектуры и алгоритма модуля.....18

3.3 Реализация модуля.....21

3.4 Тестирование модуля и рекомендации по доработке.....23

ЗАКЛЮЧЕНИЕ.....26

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....29

ПРИЛОЖЕНИЕ А.....31

ПРИЛОЖЕНИЕ Б.....37

Примечание: содержание сформировано на основе полей Word. После редактирования документа щёлкните по списку правой кнопкой мыши и выберите команду «Обновить поле» для актуализации номеров страниц.

ВВЕДЕНИЕ

Современные автоматизированные системы обработки информации стали неотъемлемой частью деятельности организаций и повседневной жизни граждан. Сетевые сервисы обмена сообщениями, электронный документооборот, банковские и государственные приложения обрабатывают значительные объёмы данных, в том числе персональных. В таких условиях надёжность программного обеспечения и корректность его поведения в нештатных ситуациях приобретают не менее важное значение, чем реализация основной функциональности [1].

Программной ошибкой принято называть дефект программы, который приводит к её некорректной работе, выдаче неверных результатов или прекращению функционирования. Полностью исключить ошибки при разработке невозможно: они возникают из-за некорректных входных данных, сбоев внешних компонентов, нарушения ограничений базы данных и допущенных разработчиком неточностей. Поэтому на практике критически важной становится не попытка устранить все ошибки как таковые, а системная организация их регистрации и обработки: приложение должно предсказуемо реагировать на исключительные ситуации, фиксировать их для последующего анализа и не раскрывать внутреннюю информацию в ответах пользователю.

С точки зрения защиты информации некорректная обработка ошибок является прямым источником угроз: подробное сообщение об исключении или стек вызовов, возвращённые клиенту, раскрывают структуру приложения и СУБД, а отсутствие журналирования лишает администратора возможности обнаружить попытки несанкционированного доступа и провести расследование инцидента. Федеральный закон от 27.07.2006 № 149-ФЗ «Об информации, информационных технологиях и о защите информации» относит обеспечение безопасности информации при её обработке в информационных системах к обязательным требованиям [1]. ГОСТ Р 51583-2014 рассматривает регистрацию событий безопасности как одну из функций защиты информации в защищённых

автоматизированных системах [6]. Персональные данные пользователей мессенджеров, к которым относятся и записи о событиях, защищаются Федеральным законом от 27.07.2006 № 152-ФЗ «О персональных данных» [2].

Таким образом, проектирование и разработка модуля регистрации и обработки программных ошибок является актуальной задачей, находящейся на пересечении разработки программных модулей и программных средств защиты информации. Объект исследования — процесс обработки программных ошибок в серверной части веб-приложения. Предмет исследования — методы и программные средства регистрации и обработки программных ошибок на платформе ASP.NET Core с применением СУБД PostgreSQL.

Целью курсового проекта является проектирование и разработка модуля регистрации и обработки программных ошибок для серверной части локального мессенджера AirGrap. Для достижения поставленной цели необходимо решить следующие задачи:

1) изучить теоретические основы регистрации и обработки программных ошибок и их роль в обеспечении безопасности автоматизированных систем;
2) рассмотреть механизмы обработки исключений и журналирования, предоставляемые платформой C#/.NET;
3) проанализировать архитектуру серверной части мессенджера AirGrap и существующее состояние модуля обработки ошибок;
4) определить требования к модулю и выполнить его проектирование;
5) реализовать модуль на языке C# с применением механизмов обработки исключений платформы .NET и СУБД PostgreSQL;
6) провести тестирование модуля и сформулировать рекомендации по его дальнейшему развитию.

Методами исследования являются анализ научной и нормативно-технической литературы, системный анализ, объектно-ориентированное проектирование, а также функциональное тестирование. Практическая часть курсового проекта выполнена на материале серверной части локального

мессенджера AirGrap, реализованной на языке C# с использованием платформы ASP.NET Core и СУБД PostgreSQL.

Курсовой проект состоит из введения, трёх параграфов основной части, заключения, списка использованных источников и приложений. В первом параграфе рассматриваются понятие и классификация программных ошибок, обработка ошибок и журналирование как элементы защиты информации, а также механизмы обработки ошибок платформы C#/.NET. Во втором параграфе приводится анализ архитектуры серверной части мессенджера AirGrap, процессов регистрации и аутентификации пользователей и текущего состояния модуля обработки ошибок. В третьем параграфе сформулированы требования к модулю, описаны его проектирование, реализация и тестирование, даны рекомендации по доработке. Практическая значимость проекта состоит в том, что разработанный модуль встроен в работающее серверное приложение и повышает его надёжность и защищённость без изменения основной функциональности.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ РЕГИСТРАЦИИ И ОБРАБОТКИ ПРОГРАММНЫХ ОШИБОК В АВТОМАТИЗИРОВАННЫХ СИСТЕМАХ

1.1 Понятие и классификация программных ошибок

Программная ошибка (дефект, в разговорной практике — «баг») представляет собой несоответствие программы её спецификации или ожидаемому поведению, которое проявляется в виде неверного результата, аварийного завершения или нарушения установленных ограничений. И. Соммервилл подчёркивает, что дефекты являются неизбежной составляющей инженерной деятельности: даже при строгих процессах разработки полный отказ от ошибок недостижим, поэтому зрелые проекты уделяют первостепенное внимание их обнаружению, обработке и анализу причин [10].

В инженерии программного обеспечения принято различать три взаимосвязанных понятия: собственно дефект (fault) — дефект в программном коде, ошибку (error) — некорректное внутреннее состояние программы, и отказ

(failure) — наблюдаемое нарушение работы системы. Один и тот же дефект может долгое время не проявляться и активироваться лишь при определённом сочетании входных данных, а сбой (кратковременное нарушение работы) при отсутствии обработки способен перерасти в устойчивый отказ системы. Из этого следует практический вывод: обработка ошибок проектируется не для отдельных случаев, а как системное свойство приложения.

Классификация ошибок может выполняться по различным основаниям. По этапу возникновения выделяют синтаксические ошибки, обнаруживаемые компилятором до запуска программы; логические и семантические ошибки, при которых синтаксически корректная программа выполняет не то, что задумано; ошибки времени выполнения (исключительные ситуации), возникающие в работающей программе из-за внешних условий или некорректных данных. По степени тяжести ошибки делят на критические, приводящие к потере данных или полной остановке системы, серьёзные, нарушающие отдельные функции, и незначительные, не влияющие на основной результат [8].

Для серверных веб-приложений особое значение имеет деление ошибок по месту возникновения. К ошибкам самого приложения относятся нарушения бизнес-логики и некорректная работа алгоритмов; к ошибкам взаимодействия — ошибки транспорта, тайм-ауты и недоступность внешних компонентов; к ошибкам уровня СУБД — нарушения ограничений целостности, ошибки синтаксиса запросов и отказы соединения. СУБД PostgreSQL каждому классу ошибок сопоставляет пятизначный код состояния SQLSTATE: например, код 23505 соответствует нарушению ограничения уникальности (unique_violation), а код 23502 — нарушению ограничения NOT NULL (not_null_violation) [12]. Именно эти коды образуют информационную основу для точной классификации ошибок базы данных в приложении.

Отдельный класс составляют дефекты самой обработки исключений: «проглатывание» ошибок пустым блоком перехвата, возврат пользователю служебной информации (стека вызовов, имён таблиц и столбцов), несогласованность кодов ответов, отсутствие записи об ошибке в журнале. Такие

дефекты не нарушают основную функциональность и потому часто остаются незамеченными, однако именно они превращают безобидный сбой в уязвимость. В рамках настоящего курсового проекта наибольший интерес представляют ошибки времени выполнения при взаимодействии приложения с СУБД и дефекты их обработки, поскольку эти категории непосредственно влияют и на надёжность, и на защищённость системы.

1.2 Обработка ошибок и журналирование как элементы защиты информации

Согласно ГОСТ Р 50922-2006, защита информации — это деятельность, направленная на предотвращение утечки защищаемой информации, а также несанкционированных и непреднамеренных воздействий на неё [4]. Классическая модель информационной безопасности оперирует тремя свойствами защищаемой информации: конфиденциальностью, целостностью и доступностью. Обработка ошибок и журналирование работают на все три свойства: корректно обработанная ошибка не раскрывает конфиденциальную внутреннюю информацию, фиксация события поддерживает целостность данных за счёт отката или безопасного восстановления, а предсказуемое поведение в нештатной ситуации сохраняет доступность сервиса.

ГОСТ Р 51583-2014 относит к функциям защиты информации в защищённых автоматизированных системах, в том числе, управление доступом и регистрацию событий безопасности [6]. Регистрация событий (журналирование, аудит) означает запись значимых для безопасности операций: попыток входа в систему, отказов в доступе, изменений данных, ошибок выполнения. Журнал событий выполняет несколько функций безопасности: обеспечивает обнаружение инцидентов по косвенным признакам (серии неудачных попыток аутентификации, всплески ошибок), создаёт доказательную базу для расследования инцидентов, позволяет восстанавливать последовательность действий нарушителя и выполняет сдерживающую роль в отношении потенциального нарушителя.

Принципы корректной обработки ошибок в защищённых системах хорошо согласуются с принципами безопасной разработки. Прежде всего это принцип безопасного отказа (fail-safe): при возникновении ошибки система должна переходить в заранее определённое безопасное состояние, а не продолжать работу в неопределённом. Второй принцип — минимального раскрытия информации: клиенту возвращается обобщённое, безопасное для публикации сообщение, тогда как полные технические детали (тип исключения, стек вызовов, параметры запроса) фиксируются только в журнале. Третий принцип — предсказуемости: для каждой категории ошибок заранее определён способ реакции, и это решение принимается на этапе проектирования, а не в момент возникновения сбоя [8].

Особого внимания заслуживает то, что сам журнал событий является объектом защиты. Записи о событиях обычно содержат персональные данные: идентификаторы пользователей, сетевые адреса, время операций. В соответствии с Федеральным законом № 152-ФЗ «О персональных данных» такие записи обрабатываются как персональные данные и требуют ограничения доступа, обеспечения целостности и установленного срока хранения [2]. Из этого вытекают практические требования к журналу: полнота записей (фиксируются как ошибки, так и значимые успешные операции), защита от несанкционированного изменения, читаемость формата и достаточность информации (метка времени, уровень события, источник, описание) для последующего анализа [13].

Таким образом, регистрация и обработка программных ошибок — это не только средство повышения надёжности, но и обязательный программный механизм защиты информации. Недооценка этого механизма приводит к типовым последствиям: раскрытию внутренней структуры системы через сообщения об ошибках, невозможности расследования инцидентов из-за отсутствия или неполноты журнала, длительным простоям из-за неуправляемых отказов.

1.3 Механизмы обработки ошибок и журналирования на платформе C#/.NET

Платформа .NET реализует механизм структурной обработки исключений. Исключение — это объект любого класса, наследующего System.Exception, который выбрасывается оператором throw при возникновении ошибочной ситуации и распространяется вверх по стеку вызовов до первого подходящего блока перехвата. Ключевые свойства класса Exception включают Message (описание ошибки), StackTrace (стек вызовов в момент возникновения), InnerException (обёрнутое исходное исключение) и Data (дополнительные сведения в виде словаря) [9]. Конструкция try/catch/finally позволяет отделить основной код от кода обработки ошибок: блоки catch перехватывают исключения по типу, при этом блоки перехвата упорядочиваются от наиболее конкретного типа к наиболее общему, а блок finally выполняется в любом случае и используется для освобождения ресурсов.

Современная практика оформления обработчиков предполагает использование фильтров исключений (условие when в блоке catch), которые позволяют перехватывать исключение не только по типу, но и по значению его свойств. Это особенно удобно при работе с СУБД: драйвер Npgsql для PostgreSQL предоставляет класс PostgresException, наследующий DbException и содержащий, помимо прочего, свойства SqlState (код состояния SQLSTATE), Detail и ConstraintName [11]. Фильтр вида catch (PostgresException ex) when (ex.SqlState == "23505") позволяет точно выделить нарушение ограничения уникальности среди прочих ошибок базы данных и среагировать на него доменным исключением приложения.

Пользовательские (доменные) исключения создаются наследованием от Exception и позволяют выразить в терминах предметной области ситуации, которые в базовых классах не различаются. Рекомендуемые правила оформления таких классов включают суффикс Exception в имени, осмысленное сообщение об ошибке по умолчанию, набор конструкторов (без параметров, с сообщением, с

сообщением и внутренним исключением) и, при необходимости, дополнительные свойства с контекстом [14]. Дж. Рихтер особо отмечает, что перехваченное исключение не следует безусловно «проглатывать»: если ошибка не может быть обработана осмысленно на данном уровне, её необходимо либо обернуть в более информативное исключение с сохранением InnerException, либо передать выше по стеку [9].

В веб-приложениях ASP.NET Core обработка ошибок распределяется по конвейеру. Глобальный уровень представлен компонентом промежуточного слоя (middleware) обработки исключений, подключаемым методом UseExceptionHandler: он перехватывает необработанные исключения и возвращает клиенту заранее определённый безопасный ответ [15]. Локальный уровень реализуется блоками try/catch в контроллерах и сервисах, где конкретные исключения отображаются на осмысленные коды HTTP-ответов: 400 (BadRequest) — некорректный запрос, 404 (NotFound) — ресурс не найден, 409 (Conflict) — конфликт состояния, 500 (InternalServerError) — внутренняя ошибка сервера. Первичная валидация входных данных до обращения к базе данных считается хорошей практикой, поскольку отклоняет заведомо некорректные запросы и снижает число ошибок на уровне СУБД.

Механизмы журналирования платформы .NET представлены интерфейсом ILogger с уровнями события от Trace до Critical, а также широким спектром сторонних библиотек (Serilog, NLog и другие). Для учебных и небольших проектов допустим собственный легковесный компонент журналирования, записывающий события в текстовый файл; при этом запись журнала должна содержать метку времени, уровень события, источник и описание, а формирование записи не должно нарушать основной ход обработки запроса [13]. Сравнение встроенных и внешних решений показывает, что выбор конкретного механизма определяется масштабом проекта: легковесное решение проще в сопровождении, внешние библиотеки предоставляют фильтрацию, ротацию и структурированные форматы.

Выводы по первому параграфу. Программные ошибки классифицируются по этапу возникновения, тяжести и месту появления; для серверных приложений ключевыми являются ошибки времени выполнения при взаимодействии с СУБД, которые в PostgreSQL кодируются стандартными кодами SQLSTATE. Обработка ошибок и журналирование являются программными механизмами защиты информации: они поддерживают конфиденциальность, целостность и доступность, обеспечивают обнаружение и расследование инцидентов. Платформа C#/.NET предоставляет полный набор средств для построения такого модуля: структурную обработку исключений, фильтры по свойствам исключения, пользовательские классы исключений, глобальный и локальный уровни обработки в ASP.NET Core и уровневое журналирование.

2 АНАЛИЗ СЕРВЕРНОЙ ЧАСТИ МЕССЕНДЖЕРА AIRGRAP И СОСТОЯНИЯ МОДУЛЯ ОБРАБОТКИ ОШИБОК

2.1 Характеристика автоматизированной системы и применённых средств разработки

AirGrap — локальный мессенджер, разрабатываемый в рамках учебной деятельности; серверная часть приложения реализована как веб-API и предоставляет функции регистрации пользователей, аутентификации по токену, создания личных диалогов и обмена сообщениями. Решение состоит из двух частей: каталога server/AirGrapServer с проектом ASP.NET Core на языке C# и каталога server/database со скриптами создания и наполнения базы данных (tables.sql, procedure.sql, functions.sql, triggers.sql и другие). Такой состав полностью охватывает серверную часть системы: клиентское приложение взаимодействует только по протоколу HTTP.

Архитектура серверной части построена по слоистому принципу, её схема приведена на рисунке 1. Уровень контроллеров ASP.NET Core (UserController, HomeController) принимает HTTP-запросы, выполняет первичную валидацию входных данных и формирует HTTP-ответы. Уровень сервисов (интерфейс IDbService и его реализация DbService) содержит бизнес-логику регистрации,

аутентификации и проверки соединения. Уровень доступа к данным (классы DataBase и ConectionString) создаёт источник NpgsqlDataSource на основе строки подключения, параметры которой загружаются из файла Datasurse/ConectionSettings.json, исключённого из системы контроля версий средствами .gitignore [11]. Нижним уровнем выступает СУБД PostgreSQL с таблицами users, sessions, conversations, conversation_participants, messages и log, хранимыми процедурами и функциями, а также расширениями pgcrypto и pg_timetable [12]. Вспомогательные компоненты — DTO (RegistrationDTO, AuntificationDTO), модуль исключений AirGrap.Exceptions и модуль журналирования AirGrap.Log — используются всеми уровнями.

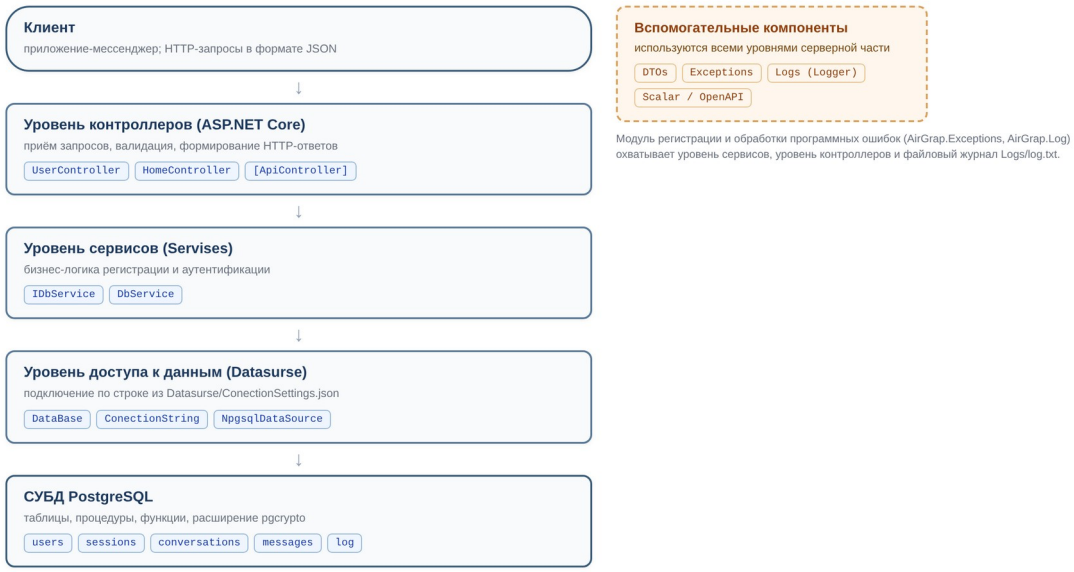


Рисунок 1 – Архитектура серверной части мессенджера AirGrap

Применённые средства разработки приведены в таблице 1.

Таблица 1 – Средства разработки серверной части AirGrap

Средство	**Назначение**
Операционная система Windows 10	среда разработки и тестирования
Платформа .NET 10 (целевая платформа net10.0)	исполнительная среда приложения
Язык программирования C#	реализация серверной части

Средство	Назначение
ASP.NET Core (Web API)	обработка HTTP-запросов, контроллеры, маршрутизация
Библиотека Npgsql 10.0.3	доступ к СУБД PostgreSQL из .NET
СУБД PostgreSQL (расширения pgcrypto, pg_timetable)	хранение данных, хеширование паролей, фоновые задания
Пакеты Microsoft.AspNetCore.OpenApi и Scalar.AspNetCore	автоматическая документация HTTP API
Редактор Visual Studio Code, система контроля версий Git	разработка и управление исходным кодом

Выбор платформы .NET и СУБД PostgreSQL обусловлен доступностью полноценных бесплатных инструментов, наличием встроенных механизмов обработки исключений и журналирования, а также поддержкой криптографических функций на стороне СУБД, что упрощает безопасную обработку паролей. Компонент Scalar предоставляет интерактивную документацию API, которая использовалась при тестировании модуля обработки ошибок.

2.2 Анализ процессов регистрации и аутентификации пользователей

Процесс регистрации начинается с POST-запроса /user/registration, тело которого десериализуется в объект RegistrationDTO с полями login, password и nickname. Контроллер UserController выполняет первичную валидацию: длина логина не менее четырёх символов, длина пароля не менее восьми символов при обязательном наличии хотя бы одной буквы, длина никнейма не менее четырёх символов. Запросы, не прошедшие валидацию, отклоняются ответом HTTP 400 без обращения к базе данных. Корректные данные передаются в метод Registration сервиса DbService, который вызывает хранимую процедуру registration(login, password, nickname); процедура вставляет запись в таблицу

users, при этом пароль хешируется алгоритмом bcrypt функцией crypt(p_password, gen_salt('bf', 10)) — в базе хранится только хеш пароля [12].

Ошибки процесса регистрации соответствуют ограничениям базы данных: уникальное ограничение столбца login порождает ошибку SQLSTATE 23505 при попытке занять существующий логин; ограничения NOT NULL порождают ошибку 23502 при пустых обязательных полях; недоступность СУБД или иные ошибки SQL завершают операцию с общими ошибками. Вне базы данных ошибочными являются невалидные входные данные, обнаруживаемые контроллером, а также внутренние ошибки кода.

Процесс аутентификации реализован POST-запросом /user/auntification и хранимой функцией auntification(login, password). Функция выполняет выборку пользователя с условием password_hash = crypt(p_password, password_hash): если пользователь не найден, возвращается NULL и контроллер отвечает HTTP 404. При успешной проверке генерируется токен encode(gen_random_bytes(32), 'base64'), создаётся запись в таблице sessions, ограничивается количество активных сессий пользователя тремя (при превышении удаляется самая старая) и обновляется поле last_login; контроллер выполняет локальное перенаправление на адрес /user/{token}. Неактивные сессии удаляются фоновым заданием планировщика pg_timetable каждые 30 минут.

Анализ обоих процессов показывает, что они насыщены точками возникновения ошибок: валидация входных данных, ограничения СУБД, отсутствие запрашиваемого объекта, отказ инфраструктуры. Именно поэтому регистрация и аутентификация выбраны в качестве прикладного контура для интеграции модуля обработки программных ошибок: они являются и наиболее употребимыми операциями мессенджера, и мишенью типовых атак (перебор логинов, подбор паролей), события которых должны попадать в журнал.

2.3 Анализ существующего состояния модуля регистрации и обработки ошибок

В серверной части AirGrap уже присутствуют зачатки модуля обработки ошибок, распределённые по трём каталогам проекта. В каталоге Exceptions определены пользовательские исключения NullInputFieldException, UserAlreadyExistsException (с дополнительным свойством Login) и RegistrationFailedException, а также пустой файл AuntificationExceptions.cs. В каталоге Logs размещён статический класс Logger с единственным методом AddLog, дописывающим текст в файл Logs/log.txt. В сервисе DbService перехват PostgresException выполняется с фильтрами по кодам SqlState: ошибка 23505 превращается в UserAlreadyExistsException, ошибка 23502 — в NullInputFieldException, прочие ошибки — в RegistrationFailedException. В контроллере UserController блоки try/catch отображают доменные исключения на коды HTTP-ответов, а при необработанном исключении вызывается Logger.AddLog и клиенту возвращается обобщённое сообщение об ошибке.

К достоинствам текущего решения следует отнести разделение ответственности между уровнями (сервис классифицирует ошибки базы данных, контроллер управляет HTTP-ответами), точную классификацию ошибок СУБД по кодам SqlState, скрытие технических деталей от клиента при необработанных ошибках и наличие хотя бы минимального журналирования. Вместе с тем анализ выявил ряд недостатков, снижающих надёжность и защищённость модуля; они систематизированы в таблице 2.

Таблица 2 – Недостатки существующего модуля обработки ошибок

№	Недостаток	Последствие
1	Файл AuntificationExceptions.cs пуст, собственных исключений аутентификации нет; при аутентификации перехватывается UserAlreadyExistsException	семантическая путаница: исключение о дубликате логина используется для ситуации входа в систему

№	Недостаток	Последствие
2	Записи журнала не содержат метки времени, уровня события, типа исключения и стека вызовов	диагностика ошибок и расследование инцидентов затруднены
3	Сервис DbService выводит ошибки в консоль (Console.WriteLine), минуя Logger	часть ошибок не попадает в файловый журнал
4	Успешные операции и события аутентификации не журналируются	отсутствует аудит безопасности, предусмотренный ГОСТ Р 51583-2014
5	Глобальный обработчик ошибок (middleware) не подключён; HomeController не обрабатывает исключения	необработанные ошибки возвращают стандартный ответ сервера с потенциальным раскрытием информации
6	Клиенту возвращается текст исключения (new { error = ex.Message })	возможное раскрытие внутренних деталей при доменных исключениях
7	Несогласованность кодов HTTP: NullInputFieldException отвечает кодом 409 Conflict	клиент не может однозначно интерпретировать причину ошибки
8	Скрипт triggers.sql ссылается на несуществующие объекты (таблицу products и функцию set_updated_at); таблица log не заполняется	механизм аудита на уровне СУБД не работает
9	Опечатки в наименованиях каталогов и файлов (Servises, Datasurse, Auntification, loger, qwery)	снижение читаемости кода, риск ошибок сопровождения

Таким образом, существующий модуль работоспособен на базовом уровне, однако его журналирующая часть неполна, контур аутентификации использует неподходящие исключения, а глобальный контур обработки ошибок отсутствует. Выявленные недостатки приняты в качестве исходных данных для проектирования доработанного модуля в третьем параграфе. Выводы по второму параграфу: серверная часть AirGrap имеет слоистую архитектуру с чётким разделением ответственности; процессы регистрации и аутентификации содержат полный набор типовых точек возникновения ошибок; текущий модуль

обработки ошибок требует доработки журналирования, аудита, обработки на глобальном уровне и устранения несогласованностей.

3 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА МОДУЛЯ РЕГИСТРАЦИИ И ОБРАБОТКИ ПРОГРАММНЫХ ОШИБОК

3.1 Требования к модулю

Проектирование модуля выполнялось как доработка существующей серверной части с сохранением её архитектуры и основного набора функций. Стадии создания и состав проектной документации определены в соответствии с ГОСТ 34.601-90 [3], а процессы анализа требований, проектирования и тестирования — с процессами жизненного цикла программных средств по ГОСТ Р ИСО/МЭК 12207-2010 [5]. Поскольку речь идёт об учебном проекте ограниченного масштаба, разработка ограничивается стадиями технического задания, эскизного и технического проектирования и рабочей документации в сокращённом объёме.

По результатам анализа, выполненного во втором параграфе, к модулю предъявлены функциональные и нефункциональные требования, приведённые в таблице 3.

Таблица 3 – Требования к модулю регистрации и обработки программных ошибок

Код	Требование	Пояснение
Ф1	Классификация ошибок СУБД	ошибки PostgreSQL распознаются по кодам SqlState и преобразуются в доменные исключения
Ф2	Журналирование необработанных ошибок	необработанные исключения фиксируются записью в файловом журнале
Ф3	Безопасные HTTP-ответы	каждая категория ошибок отображается на предопределённый код ответа без раскрытия внутренних деталей

Код	Требование	Пояснение
Ф4	Первичная валидация входных данных	некорректные запросы отклоняются до обращения к базе данных
Ф5	Понятное описание ошибки	сообщение, возвращаемое клиенту, достаточно для исправления запроса
НФ1	Расширяемость	новые классы исключений добавляются без изменения существующих обработчиков
НФ2	Сохранение работоспособности	формирование записи журнала не прерывает обработку запроса
НФ3	Единообразие наименований	имена файлов и компонентов согласованы между собой
НФ4	Минимальные зависимости	используются только средства платформы и уже подключённые пакеты

Требования Ф1–Ф5 охватывают контур обработки ошибок, а требования НФ1–НФ4 задают инженерные ограничения, соответствующие масштабу учебного проекта. Явное выделение требований позволило при реализации контролировать полноту решения и вернуться к каждому пункту при тестировании.

3.2 Проектирование архитектуры и алгоритма модуля

Модуль состоит из трёх связанных компонентов. Первый компонент — иерархия пользовательских исключений в пространстве имён AirGrap.Exceptions: классы, наследующие System.Exception и выражающие предметные ситуации — занятый логин, пустое обязательное поле, общая неудача операции. Второй компонент — компонент журналирования AirGrap.Log: статический класс Logger, доступный из любого места приложения без создания экземпляров и записывающий события в файловый журнал Logs/log.txt. Третий компонент — обработчики ошибок на уровнях сервиса DbService (классификация ошибок СУБД) и контроллера UserController

(перехват исключений, формирование HTTP-ответов, вызов журнала). Схема взаимодействия компонентов приведена на рисунке 2.

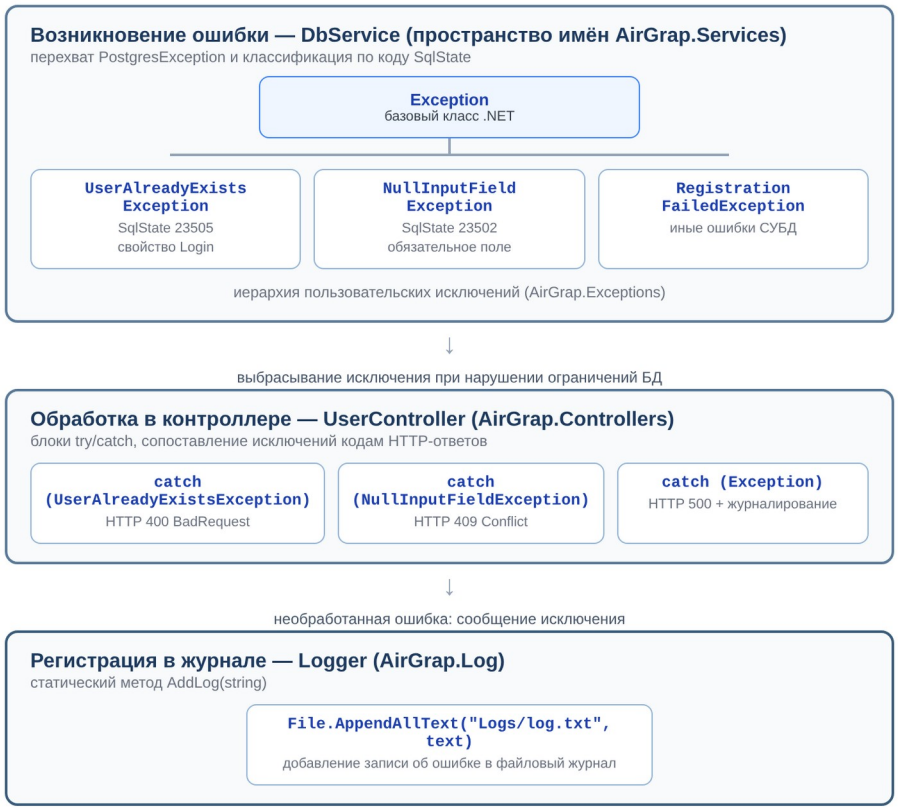


Рисунок 2 – Схема взаимодействия компонентов модуля регистрации и обработки ошибок

Разграничение ответственности между компонентами выбрано следующим образом: уровень сервиса знает коды СУБД и порождает доменные исключения; уровень контроллера знает протокол HTTP и отображает исключения на коды ответов; компонент журналирования ничего не знает ни о СУБД, ни о протоколе и выполняет только запись текста события. Такое решение соответствует рекомендациям по обработке исключений платформы .NET [14] и позволяет добавлять новые классы ошибок без изменения компонентов журналирования (требование НФ1).

Обработка ошибки при регистрации пользователя выполняется по алгоритму из четырёх этапов, показанному на рисунке 3. На этапе валидации контроллер отклоняет заведомо некорректные запросы ответом HTTP 400, не

обращаясь к базе данных (требование Ф4). На этапе выполнения сервис вызывает хранимую процедуру и при нарушении ограничений получает PostgresException. На этапе классификации код SqlState сопоставляется с доменным исключением (требование Ф1). На этапе формирования ответа контроллер возвращает клиенту предопределённый код и безопасное сообщение (требования Ф3, Ф5), а необработанные исключения предварительно фиксируются в журнале (требование Ф2).



Рисунок 3 – Алгоритм обработки ошибки при регистрации пользователя

3.3 Реализация модуля

Иерархия пользовательских исключений реализована в каталоге Exceptions проекта; её фрагмент приведён в листинге 1. Класс UserAlreadyExistsException помимо унаследованных членов содержит свойство Login с занятым логином, что делает сообщение об ошибке конкретным и полезным для клиента; классы NullInputFieldException и RegistrationFailedException формируют фиксированные сообщения об ошибке в конструкторе, поэтому при выбрасывании не требуют передачи параметров.

Листинг 1 – Фрагмент файла UserExceptions.cs (полный текст — в приложении А)

```
namespace AirGrap.Exceptions;

public class NullInputFieldException : Exception
{
    public NullInputFieldException()
        : base("There is no mandatory value"){ }
}
public class UserAlreadyExistsException : Exception
{
    public string Login { get; }
    public UserAlreadyExistsException(string _login)
        : base($"login '{_login}' already exists") => Login = _login;
}
public class RegistrationFailedException : Exception
{
    public RegistrationFailedException()
        : base("registration failed try again"){ }
}
```

Компонент журналирования реализован статическим классом Logger (листинг 2). Метод AddLog дописывает полученный текст в конец файла Logs/log.txt; поскольку класс статический, обращение к нему возможно из любого уровня приложения без внедрения зависимостей, что соответствует требованию НФ4 о минимальных зависимостях.

Листинг 2 – Файл loger.cs компонента журналирования

```
namespace AirGrap.Log;
public static class Logger
{
    public static void AddLog(string text)
```

```
    {
        File.AppendAllText("Logs/log.txt", text);
    }
}
```

Ключевая часть обработки ошибок сосредоточена в сервисе DbService (листинг 3). Команда создаётся методом CreateCommand источника NpgsqlDataSource и вызывает хранимую процедуру registration с параметрами; блочные фильтры when позволяют перехватить PostgresException строго по коду SqlState и выбросить соответствующее доменное исключение. Фильтр по коду 23505 передаёт в конструктор исключения занятый логин, фильтр по коду 23502 сигнализирует о пустом обязательном поле, а последний фильтр оборачивает любую прочую ошибку СУБД в RegistrationFailedException, не раскрывая её деталей наверх.

Листинг 3 – Метод Registration сервиса DbService с классификацией ошибок СУБД

```
public async Task Registration(string login, string Password, string nicname)
{
    try
    {
        await using var cmd = _dataBase.CreateCommand("call
registration($1,$2,$3);");
        cmd.Parameters.AddWithValue(login);
        cmd.Parameters.AddWithValue(Password);
        cmd.Parameters.AddWithValue(nicname);
        await cmd.ExecuteNonQueryAsync();
    }
    catch (PostgresException ex) when (ex.SqlState == "23505")
    {
        throw new UserAlreadyExistsException(login);
    }
    catch (PostgresException ex) when (ex.SqlState == "23502")
    {
        throw new NullInputFieldException();
    }
    catch (PostgresException ex)
    {
        throw new RegistrationFailedException();
    }
}
```

На уровне контроллера (листинг 4) доменные исключения отображаются на коды HTTP-ответов: занятый логин — 400 BadRequest с понятным клиенту описанием, пустое поле — 409 Conflict, любые прочие исключения — 500 с

обобщённым сообщением. Перед возвратом ответа 500 в журнал записывается текст исключения вызовом Logger.AddLog (требование Ф2). Внедрение сервиса выполняется через механизм зависимостей ASP.NET Core: в Program.cs регистрируются NpgsqlDataSource и пара «интерфейс — реализация» (AddSingleton<IDbService, DbService>), благодаря чему контроллер получает IDbService автоматически и не зависит от деталей доступа к данным.

Листинг 4 – Обработка исключений в методе Registration контроллера UserController

```
try
{
    await _dbservise.Registration(body.login, body.password, body.nickname);
    return Ok();
}
catch (UserAlreadyExistsException ex)
{
    return BadRequest(new { error = ex.Message });
}
catch (NullInputFieldException ex)
{
    return Conflict(new { error = ex.Message });
}
catch (Exception ex)
{
    Logger.AddLog(String.Concat(ex.Message, ex.Data.ToString()));
    return StatusCode(500, new { error = "Internal server error pliase try
again" });
}
```

Полные листинги всех реализованных компонентов, включая Program.cs, контроллеры, файлы доступа к данным и DTO, приведены в приложении А; скрипты базы данных, участвующие в обработке ошибок, — в приложении Б.

3.4 Тестирование модуля и рекомендации по доработке

Тестирование выполнялось методом функционального (чёрного ящика) тестирования HTTP-интерфейса: сервер запускался локально, запросы формировались через интерактивную документацию Scalar, проверялись код ответа, тело ответа и содержимое файла журнала. Сценарии покрывают все требования Ф1–Ф5 и обе основные операции системы. Результаты приведены в таблице 4.

Таблица 4 – Результаты функционального тестирования модуля

№	Сценарий	Ожидаемый результат	Результат
1	Регистрация с корректными данными	HTTP 200, запись создана в таблице users	соответствует
2	Регистрация с логином короче 4 символов	HTTP 400, сообщение о коротком логине	соответствует
3	Регистрация с паролем короче 8 символов	HTTP 400, сообщение о коротком пароле	соответствует
4	Регистрация с паролем без букв	HTTP 400, сообщение об отсутствии букв	соответствует
5	Повторная регистрация существующего логина	HTTP 400, сообщение вида «login '...' already exists»	соответствует
6	Аутентификация с корректными данными	токен, перенаправление на /user/{token}	соответствует
7	Аутентификация с неверным паролем	HTTP 404 «user not found»	соответствует
8	Регистрация при остановленной СУБД	HTTP 500 с обобщённым сообщением, запись в журнал	соответствует

По всем восьми сценариям фактическое поведение совпало с ожидаемым. Дополнительно проверялась работа журнала: после сценария 8 в файле Logs/log.txt появлялась запись с текстом исключения, что подтверждает выполнение требования Ф2. Таким образом, требования Ф1–Ф5 в реализованной версии модуля выполнены.

Вместе с тем тестирование подтвердило и ограничения, выявленные при анализе в параграфе 2: записи журнала не содержат метки времени и уровня события, часть ошибок сервиса не попадает в журнал, контур аудита на уровне СУБД не задействован. На основании этого сформулированы рекомендации по дальнейшей доработке модуля:

1) подключить глобальный обработчик исключений UseExceptionHandler, перехватывающий ошибки всех контроллеров, включая HomeController, и возвращающий единый безопасный ответ [15];

2) расширить формат записи журнала: добавить метку времени, уровень события, тип исключения и стек вызовов; в перспективе перейти на интерфейс ILogger или библиотеку Serilog [13];
3) журналировать в DbService через Logger вместо Console.WriteLine и дополнительно фиксировать значимые успешные операции (успешная регистрация, вход в систему) для ведения аудита;
4) реализовать классы исключений аутентификации в файле AuntificationExceptions.cs (например, InvalidCredentialsException) и убрать из контроллера аутентификации перехват UserAlreadyExistsException;
5) согласовать коды HTTP-ответов: NullInputFieldException отвечать кодом 400, а не 409;
6) задействовать таблицу log: реализовать в триггерах регистрации событий аудита и исправить скрипт triggers.sql, ссылающийся на несуществующие объекты;
7) хранить параметры подключения через механизм пользовательских секретов или переменные окружения вместо файла с настройками;
8) унифицировать наименования каталогов и файлов (Services, DataSource, Authentication, Logger, query).

Выводы по третьему параграфу. Модуль регистрации и обработки программных ошибок спроектирован по требованиям Ф1–Ф5 и НФ1–НФ4, реализован в составе серверной части AirGrap и подтверждён функциональным тестированием: все восемь сценариев дали ожидаемые результаты, журнал фиксирует необработанные ошибки. Сформулированные рекомендации образуют план дальнейшей доработки, не затрагивающий основную функциональность системы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта достигнута поставленная цель: спроектирован и разработан модуль регистрации и обработки программных ошибок для серверной части локального мессенджера AirGrap. Последовательно решены все задачи, сформулированные во введении.

По первой задаче изучены теоретические основы предметной области. Раскрыты понятия программной ошибки, сбоя и отказа, приведена классификация ошибок по этапу возникновения, степени тяжести и месту появления. Показано, что для серверных веб-приложений ключевыми являются ошибки времени выполнения при взаимодействии с СУБД, которые в PostgreSQL кодируются стандартными кодами состояния SQLSTATE. Обоснована роль обработки ошибок и журналирования как программных механизмов защиты информации, поддерживающих конфиденциальность, целостность и доступность данных, с опорой на требования Федеральных законов № 149-ФЗ и № 152-ФЗ и стандартов ГОСТ Р 50922-2006 и ГОСТ Р 51583-2014.

По второй задаче рассмотрены механизмы платформы C#/.NET: структурная обработка исключений try/catch/finally, фильтры исключений, класс PostgresException с кодами SqlState, правила построения пользовательских классов исключений, глобальный и локальный уровни обработки ошибок в ASP.NET Core, уровневое журналирование. Полученные знания позволили выбрать решения, соответствующие масштабу учебного проекта.

По третьей задаче выполнен анализ серверной части мессенджера AirGrap. Установлено, что приложение имеет слоистую архитектуру с разделением на уровни контроллеров, сервисов, доступа к данным и СУБД PostgreSQL; проанализированы процессы регистрации и аутентификации, насыщенные точками возникновения ошибок. Анализ существующего модуля обработки ошибок выявил девять недостатков, среди которых — пустой файл исключений аутентификации, неполнота записей журнала, вывод ошибок сервиса в консоль

вместо журнала, отсутствие глобального обработчика исключений и неработающий механизм аудита на уровне СУБД.

По четвёртой задаче сформулированы функциональные и нефункциональные требования к модулю, выполнено его проектирование: определены три компонента (иерархия пользовательских исключений, компонент журналирования Logger, обработчики на уровнях сервиса и контроллера), построены схема взаимодействия компонентов и алгоритм обработки ошибки при регистрации пользователя.

По пятой задаче модуль реализован на языке C#: пользовательские исключения UserAlreadyExistsException, NullInputFieldException и RegistrationFailedException; статический класс Logger, записывающий события в файловый журнал; классификация ошибок СУБД по кодам SqlState в сервисе DbService; обработка исключений в контроллере UserController с формированием предопределённых HTTP-ответов и вызовом журнала. Полные листинги приведены в приложении А.

По шестой задаче проведено функциональное тестирование модуля: восемь сценариев охватили валидацию, классификацию ошибок СУБД, формирование HTTP-ответов и работу журнала; фактическое поведение во всех сценариях совпало с ожидаемым. По итогам анализа и тестирования сформулированы рекомендации по доработке, включающие подключение глобального обработчика исключений, расширение формата журнала, ведение аудита успешных операций, реализацию исключений аутентификации, согласование кодов HTTP-ответов, задействование таблицы аудита log на уровне СУБД и унификацию наименований.

Таким образом, задачи курсового проекта решены полностью, цель достигнута. Практическая значимость проекта состоит в том, что разработанный модуль встроен в работающее серверное приложение: операции регистрации и аутентификации стали предсказуемо обрабатывать ошибки, не раскрывать внутреннюю структуру системы и фиксировать нештатные ситуации в журнале. Сформулированные рекомендации могут быть использованы при дальнейшем

развитии мессенджера AirGrap и при разработке других приложений на платформе ASP.NET Core.

Дата окончания работы: «___» _____ 2026 г.

Подпись студента: _____ / Цоколов Д. Д.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Законы. Об информации, информационных технологиях и о защите информации [Текст] : Федеральный закон № 149-ФЗ : принят Государственной Думой 8 июля 2006 года. – Москва : Собрание законодательства Российской Федерации, 2006. – № 31 (ч. 1). – Ст. 3448.
2. Российская Федерация. Законы. О персональных данных [Текст] : Федеральный закон № 152-ФЗ : принят Государственной Думой 8 июля 2006 года. – Москва : Собрание законодательства Российской Федерации, 2006. – № 31 (ч. 1). – Ст. 3451.
3. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания [Текст]. – Москва : Стандартинформ, 2009.
4. ГОСТ Р 50922-2006. Защита информации. Основные термины и определения [Текст]. – Москва : Стандартинформ, 2008.
5. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств [Текст]. – Москва : Стандартинформ, 2011.
6. ГОСТ Р 51583-2014. Защита информации. Порядок создания автоматизированных систем в защищённом исполнении. Общие положения [Текст]. – Москва : Стандартинформ, 2015.
7. Макконнелл, С. Совершенный код. Мастер-класс [Текст] / С. Макконнелл ; пер. с англ. – Санкт-Петербург : Питер ; Москва : Русская редакция, 2005. – 1024 с.
8. Мартин, Р. Чистый код. Создание, анализ и рефакторинг [Текст] / Р. Мартин ; пер. с англ. – Санкт-Петербург : Питер, 2018. – 464 с.
9. Рихтер, Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C# [Текст] / Дж. Рихтер ; пер. с англ. – 2-е изд. – Санкт-Петербург : Питер, 2014. – 896 с.

10. Соммервилл, И. Инженерия программного обеспечения [Текст] / И. Соммервилл ; пер. с англ. – 9-е изд. – Москва : Вильямс, 2011. – 624 с.
11. Npgsql Documentation [Электронный ресурс] // Npgsql : [сайт]. – URL: https://www.npgsql.org/doc/ (дата обращения: 12.11.2026).
12. PostgreSQL Documentation [Электронный ресурс] // PostgreSQL : [сайт]. – URL: https://www.postgresql.org/docs/ (дата обращения: 12.11.2026).
13. Ведение журнала в .NET Core и ASP.NET Core [Электронный ресурс] // Microsoft Learn : [сайт]. – URL: https://learn.microsoft.com/ru-ru/dotnet/core/extensions/logging (дата обращения: 12.11.2026).
14. Обработка и создание исключений в C# [Электронный ресурс] // Microsoft Learn : [сайт]. – URL: https://learn.microsoft.com/ru-ru/dotnet/csharp/fundamentals/exceptions/ (дата обращения: 12.11.2026).
15. По промежуточного слоя ASP.NET Core [Электронный ресурс] // Microsoft Learn : [сайт]. – URL: https://learn.microsoft.com/ru-ru/aspnet/core/fundamentals/middleware/ (дата обращения: 12.11.2026).

ПРИЛОЖЕНИЕ А

Листинги компонентов серверной части приложения

A.1 Файл Program.cs — точка входа серверной части

```
using Microsoft.AspNetCore.OpenApi;
using AirGrap.Services;
using AirGrap.Datasurse;
using Scalar.AspNetCore;
namespace AirGrap;
class Program
{
    static void Main(string[] args)
    {
        DataBase dataBase = new();

        var builder = WebApplication.CreateBuilder(args);
        builder.Services.AddSingleton(dataBase.dataSource);
        builder.Services.AddSingleton<IDbService, DbService>();
        builder.Services.AddOpenApi();
        builder.Services.AddControllers();
        var app = builder.Build();
        app.MapControllers();
        app.MapOpenApi();
        app.MapScalarApiReference();
        app.Run();
    }

}
```

A.2 Файл Controllers/UserController.cs

```
using Microsoft.AspNetCore.Mvc;
using AirGrap.Exceptions;
using AirGrap.Log;
using AirGrap.DTOs;


namespace AirGrap.Controllers;


[ApiController]
[Route("user")]
public class UserController(IDbService _dbservise) : ControllerBase
{
  [HttpPost("registration")]
  public async Task<IActionResult> Registration([FromBody] RegistrationDTO body)
  {
    if (body.login == null  || body.login.Length < 4) return BadRequest("login
is too short < 4 characters");
    if (body.password == null  || body.password.Length < 8) return
BadRequest("the password is too short < 8 characters");
    else if (!body.password.Any(char.IsLetter)) return BadRequest("The password
contains no letters.");
    if (body.nickname == null  || body.nickname.Length < 4) return
BadRequest("nickname is too short < 4 characters");
    try
    {
```

```
      await _dbservise.Registration(body.login,body.password,body.nickname);
      return Ok();
    }
    catch (UserAlreadyExistsException ex)
    {
      return BadRequest(new {error = ex.Message});
    }
    catch (NullInputFieldException ex)
    {
      return Conflict(new {error = ex.Message});
    }
    catch(Exception ex)
    {
      Logger.AddLog(String.Concat(ex.Message,ex.Data.ToString()));
      return StatusCode(500, new { error = "Internal server error pliase try
again"});
    }
  }
  [HttpPost("auntification")]
  public async Task<IActionResult> Auntification(AuntificationDTO body)
  {
    if (body.login == null) return BadRequest("login is null");
    if (body.password == null ) return BadRequest("the password is null");
    try
    {
      string? token = await _dbservise.Auntification(body.login,body.password);
      return (token == null || token == String.Empty)? NotFound("user not
found"): LocalRedirect($"/User/{token}");
    }
    catch (UserAlreadyExistsException ex)
    {
      return Conflict(new {error = ex.Message});
    }
    catch(Exception ex)
    {
      Logger.AddLog(String.Concat(ex.Message,ex.Data.ToString()));
      return StatusCode(500, new { error = "Internal server error pliase try
again"});
    }
  }
  [HttpGet("{token}")]
  public IActionResult Authorized(string token)
  {
    return Ok("Auntification ok");
  }
}
```

A.3 Файл Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
namespace AirGrap.Controllers;


[ApiController]
[Route("[controller]")]
public class HomeController : ControllerBase
{
    private readonly IDbService _DbServise;


    public HomeController(IDbService DbServise)
    {
        _DbServise = DbServise;
    }
```

```
    [HttpGet("/")]
    public IActionResult Index()
    {
        return Ok("Server is Actiive");
    }
    [HttpGet("/test")]
    public async Task<IActionResult> TestConect()
    {
        return Ok((await _DbServise.CheckConnectionAsync()).ToString());
    }
    [HttpGet("/red")]
    public async Task<IActionResult> red()
    {
        return Redirect("/test");
    }
}
```

A.4 Файл Servises/IDBServise.cs

```
using Npgsql;
using AirGrap.Datasurse;
public interface IDbService
{
    public Task<ServerStatus> CheckConnectionAsync();
    public Task Registration(string login,string Password,string nicname);
    public Task<string?> Auntification(string login,string Password);
}
```

A.5 Файл Servises/DBServise.cs

```
using Npgsql;
using AirGrap.Datasurse;
using AirGrap.Exceptions;
namespace AirGrap.Services;

public class DbService (NpgsqlDataSource _dataBase) : IDbService
{

    public async Task<ServerStatus> CheckConnectionAsync()
    {
        try
        {
            await using var cmd = _dataBase.CreateCommand("SELECT 1");
            var raw = await cmd.ExecuteScalarAsync();
            return raw is int i && i == 1 ? ServerStatus.OK :
ServerStatus.Error;
        }
        catch (Exception ex)
        {
            Console.WriteLine($"DB check failed: {ex.Message}");
            return ServerStatus.NoConection;
        }
    }
    public async Task Registration(
        string login,string Password,string nicname)
    {


        try
        {
            await using var cmd = _dataBase.CreateCommand("call
registration($1,$2,$3);");
```

```
            cmd.Parameters.AddWithValue(login);
            cmd.Parameters.AddWithValue(Password);
            cmd.Parameters.AddWithValue(nicname);
            await cmd.ExecuteNonQueryAsync();
        }
        catch (PostgresException ex) when (ex.SqlState == "23505")
        {
            throw new UserAlreadyExistsException(login);
        }
        catch (PostgresException ex) when (ex.SqlState == "23502")
        {
            throw new NullInputFieldException();
        }
        catch (PostgresException ex)
        {
            Console.WriteLine($"Registration failed: {ex.Message}");
            throw new RegistrationFailedException();
        }
    }
    public async Task<string?> Auntification(string login,string Password)
    {
        try
        {
            await using var cmd = _dataBase.CreateCommand("select
auntification($1,$2)");
            cmd.Parameters.AddWithValue(login);
            cmd.Parameters.AddWithValue(Password);
            string? token = (await cmd.ExecuteScalarAsync() ??
String.Empty).ToString();
            return token;
        }

        catch (PostgresException ex) when (ex.SqlState == "23505")
        {
            throw new UserAlreadyExistsException(login);
        }
        catch (PostgresException ex)
        {
            Console.WriteLine($"Registration failed: {ex.Message}");
            throw new RegistrationFailedException();
        }
    }

}
```

A.6 Файл Datasurse/DataBase.cs

```
using System.Text.Json;
using Npgsql;



namespace AirGrap.Datasurse;

public class DataBase
{
    public NpgsqlDataSource dataSource;
    public DataBase()
    {
        ConectionString? ConInf = null;
        if (File.Exists("Datasurse/ConectionSettings.json"))
        {
```

```
            var jsonInput =
File.ReadAllText("Datasurse/ConectionSettings.json");
            ConInf = JsonSerializer.Deserialize<ConectionString>(jsonInput);
        }
        if (ConInf == null) ConInf = new();

        string connectionString =
$"Host={ConInf.Host};Port={ConInf.Port};Database={ConInf.Database};Username={Con
Inf.Username};Password={ConInf.Password}";
        dataSource = NpgsqlDataSource.Create(connectionString);
    }



}
```

A.7 Файл Datasurse/ConectionString.cs

```
using System;

namespace AirGrap.Datasurse;

public class ConectionString
{
    public string Host { get; set; } = "localhost" ;
    public int Port { get; set; } = 5432;
    public string Database { get; set; } = "airgrap";
    public string Username { get; set; } = "postgres";
    public string Password { get; set; } = "postgre";

}
```

A.8 Файл DTOs/RegistrationDTO.cs

```
namespace AirGrap.DTOs;
public class RegistrationDTO
{
    public string? login {get;set;}
    public string? password {get;set;}
    public string? nickname {get;set;}
}
```

A.9 Файл DTOs/AuntificationDTO.cs

```
namespace AirGrap.DTOs;
public class AuntificationDTO
{
    public string? login {get;set;}
    public string? password {get;set;}
}
```

A.10 Файл Exceptions/UserExceptions.cs

```
namespace AirGrap.Exceptions;
public class NullInputFieldException : Exception
{
    public NullInputFieldException()
        : base("There is no mandatory value"){}
}
```

```
public class UserAlreadyExistsException : Exception
{
    public string Login { get; }
    public UserAlreadyExistsException(string _login)
        : base($"login '{_login}' already exists") => Login = _login;
}
```

A.11 Файл Exceptions/RegistrationExceptions.cs

```
namespace AirGrap.Exceptions;

public class RegistrationFailedException : Exception
{
    public RegistrationFailedException()
        : base("registration failed try again"){}
}
```

A.12 Файл Logs/loger.cs

```
namespace AirGrap.Log;
public static class Logger
{
    public static void AddLog(string text)
    {
        File.AppendAllText("Logs/log.txt",text);
    }
}
```

ПРИЛОЖЕНИЕ Б

SQL-скрипты базы данных мессенджера

Б.1 Файл database/tables.sql — структура таблиц

```
drop table if exists log;
drop table if exists messages;
drop table if exists conversation_participants;
drop table if exists conversations;
drop table if exists sessions;
drop table if exists users;
CREATE EXTENSION IF NOT EXISTS pgcrypto;


create table users (
      id bigserial primary key,
    login varchar unique not null,
      password_hash text not null,
      nickname varchar not null ,
      last_login timestamptz ,
      account_type char(1) not null default 'U' check (account_type in
('U','A')),
    created_at timestamptz  not null default now(),
      settings jsonb not null default '{}'::jsonb
);



create table conversations  (
      id bigserial primary key,
      name varchar,
      type char(1) not null check (type in ('D','G')),
      created_at timestamptz  not null default now(),
      updated_at timestamptz not null default now(),
      check (type = 'D' or name is not null)
);
create index idx_conversations_type on conversations(type);
create index idx_conversations_updated on conversations(updated_at desc);



create table conversation_participants (
      conversation_id BIGINT not null references conversations(id) on delete
cascade,
    user_id BIGINT not null references users(id) on delete cascade,
      user_role char(1) not null default 'U' check (user_role in ('U','A','B')),
      last_read_id BIGINT,
    joined_at timestamptz  not null default now(),
    PRIMARY KEY (conversation_id, user_id)
);
create index idx_participants_user on conversation_participants(user_id);
create index idx_participants_conv on
conversation_participants(conversation_id);



create table messages (
    id BIGSERIAL PRIMARY KEY,
```

```
    conversation_id BIGINT not null references conversations(id) on delete
cascade,
    user_id BIGINT references users(id) on delete set null,
    message text,
      reply_to_id BIGINT references messages(id) on delete set null,
    created_at timestamptz not null default now(),
    edited_at timestamptz
);
create index idx_messages_conv_id on messages(conversation_id, id desc);



create table sessions (
    user_id bigint not null references users(id) on delete cascade,
      token text primary key,
      created_at timestamptz not null default now(),
    last_activity_at timestamptz  not null default now()
);
create index idx_sessions_user on sessions(user_id);
create index idx_sessions_last_activity on sessions(last_activity_at);


create table log (
      table_name varchar not null,
      event_type char(1) check (event_type in ('D','U','I')),
      old_value jsonb,
    new_value jsonb,
      user_id bigint,
      created_at timestamptz not null default now()
);
```

Б.2 Файл database/procedure.sql — хранимые процедуры

```
drop PROCEDURE if exists registration;
drop PROCEDURE if exists send_message(text,bigint,text);
drop PROCEDURE if exists send_message(text,bigint,text,bigint);


CREATE PROCEDURE registration(_login varchar, p_password varchar,_nickname
varchar)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO users (login, password_hash,nickname) VALUES (_login,
crypt(p_password,gen_salt('bf',10)),_nickname);
END;
$$;


create PROCEDURE send_message (i_token text,i_conversation_id bigint,i_message
text)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
declare
p_user_id bigint;
BEGIN
       p_user_id := (select user_id from sessions where token = i_token);
       if p_user_id is null
            then raise exception 'session is not found'; end if ;
       if not exists (select * from conversation_participants as c where
c.conversation_id = i_conversation_id and c.user_id = p_user_id) then
            raise exception 'conversation is not found'; end if ;
```

```
       insert into messages(conversation_id,user_id,message) values
(i_conversation_id,p_user_id,i_message);
END;
$$;


create PROCEDURE send_message (i_token text,i_conversation_id bigint,i_message
text,i_reply_to_id bigint)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
declare
p_user_id bigint;
BEGIN
       p_user_id := (select user_id from sessions where token = i_token);
       if p_user_id is null
            then raise exception 'session is not found'; end if ;
       if not exists (select * from conversation_participants as c where
c.conversation_id = i_conversation_id and c.user_id = p_user_id) then
            raise exception 'conversation is not found'; end if ;
       if not exists (select * from messages as m where m.id = i_reply_to_id)
then
            raise exception 'conversation is not found'; end if ;
       insert into messages(conversation_id,user_id,message,reply_to_id) values
(i_conversation_id,p_user_id,i_message,i_reply_to_id);
END;
$$;
```

Б.3 Файл database/functions.sql — хранимые функции

```
drop function if exists auntification;
drop function if exists create_direct_conversation;


create function auntification (_login varchar,p_password varchar)
returns TEXT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
DECLARE
    v_token TEXT;
      v_user_id bigint;
BEGIN
    v_user_id := (select id from users where login = _login and password_hash =
crypt(p_password, password_hash));
      if v_user_id IS NULL THEN  RETURN NULL;END IF;
      v_token := encode(gen_random_bytes(32), 'base64');
      INSERT INTO sessions(user_id,token) values (v_user_id,v_token);
      if (select count(user_id) from sessions where user_id = v_user_id group by
user_id) > 3 then
            delete from sessions where created_at = (select min(created_at) from
sessions where user_id = v_user_id) and user_id = v_user_id;
            end if;
      update users set last_login = now() where users.id = v_user_id ;
      return v_token;
END;
$$;



create function create_direct_conversation(i_token text ,interlocutor_id bigint)
returns bigint
```

```
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = public
AS $$
declare
      p_user_id bigint;
      p_conversations_id bigint;
BEGIN
      p_user_id := (select s.user_id from sessions as s where i_token =
s.token);
      if p_user_id is null then raise exception 'invalid token'; end if;
      if not exists (select 1 from users where id = interlocutor_id) then raise
exception 'interlocutor not found';end if;
      p_conversations_id := (select c.id from conversations c
            join conversation_participants c0 ON c0.conversation_id = c.id AND
c0.user_id = p_user_id
      join conversation_participants c1 ON c1.conversation_id = c.id AND
c1.user_id = interlocutor_id
      where c.type = 'D');
      if p_conversations_id is null then
            insert INTO conversations (type)
            values ('D') returning id into p_conversations_id;
            insert into conversation_participants(conversation_id,user_id)
                  values (p_conversations_id,p_user_id);
            insert into conversation_participants(conversation_id,user_id)
                  values (p_conversations_id,interlocutor_id);
      end if;
      return p_conversations_id;
END;
$$;
```